Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №1
по дисциплине
«Администрирование и программирование распределенных приложений»
Вариант 2

Выполнила: ст. гр. 320604
К.А. Хаджинова

Проверила: Ю.О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 ХОД РАБОТЫ 4
2.1 Постановка задачи 4
2.2 Теоретические сведения 4
2.3 Подготовка среды 4
2.4 Индивидуальное задание и исходные данные 5
2.5 Программа 5
2.6 Результаты работы 6
3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 9
ВЫВОДЫ 10
ПРИЛОЖЕНИЕ А (ОБЯЗАТЕЛЬНОЕ) ЛИСТИНГИ ПРОГРАММНОГО КОДА 11

1 ЦЕЛЬ РАБОТЫ

Изучить механизмы работы с *Web*-ресурсами в языке *Java*: получение содержимого ресурса по *URL* с помощью потоковых классов, программный запуск браузера, получение списка файлов каталога и построение оконного приложения на основе библиотеки *AWT*. Научиться выполнять индивидуальное задание варианта 2 – поиск названия лекарства по симптомам в файле-справочнике.

2 ХОД РАБОТЫ

Работа выполнена на языке *Java* 17 с использованием библиотеки *AWT*. Полный текст программы приведен в приложении А. Ниже последовательно описаны постановка задачи, теоретическая основа, реализация заданий и результаты проверки.

2.1 Постановка задачи

Для изучения работы с *Web*-ресурсами в рамках работы требуется выполнить следующие задачи:

1 создать оконное приложение на основе заготовки из методических указаний;

2 прочитать содержимое файла-справочника через *URL* и потоковые классы;

3 реализовать индивидуальное задание: по введенным симптомам найти название лекарства;

4 вывести результат пользователю в окне приложения.

2.2 Теоретические сведения

Работа посвящена поиску информационных документов по набору ключевых слов. Приложение вводит ключевые слова, находит документ, соответствующий (релевантный) введенному набору, и открывает его в браузере. Источником данных служат ресурсы, адресуемые *URL* (*Uniform Resource Locator*): локальные файлы и документы сети.

В *Java* для чтения ресурса по *URL* используются классы *URL* и *URLConnection*: метод *openConnection*() создает соединение, а метод *getInputStream*() возвращает входной поток, который оборачивается потоковыми классами *InputStreamReader* и *BufferedReader* для построчного чтения текста. Список файлов каталога возвращает метод *File.listFiles*(). Графический интерфейс строится на классах *AWT*: окно *Frame*, кнопки *Button*, текстовая область *TextArea*, а действия кнопок обрабатывает интерфейс *ActionListener*. Открыть документ в браузере по умолчанию позволяет метод *Desktop.getDesktop*().*browse*(*uri*).

2.3 Подготовка среды

Приложение написано на *Java* и скомпилировано *JDK* 17 (*Zulu* 17.0.15) командой *javac -encoding UTF-8 -d out src\web_res_lab\Web_res_lab.java*.

Запуск выполняется из каталога лабораторной работы командой *java -Dfile.encoding=UTF-8 -cp out web_res_lab.Web_res_lab*. В каталоге находятся файл *medicines.txt* и папка *source_html* с *html*-документами.

2.4 Индивидуальное задание и исходные данные

Вариант 2: выполнить поиск названия лекарства по набору из одного-трех симптомов. Список лекарств и симптомов представлен в текстовом файле. Набор данных создан для лабораторной работы и носит учебный характер (не является медицинской рекомендацией); он должен быть согласован с преподавателем. Каждая строка файла *medicines.txt* имеет формат «Название | симптом, симптом, ...». Для каждого лекарства подготовлен *html*-документ в папке *source_html.* Состав набора данных приведен в таблице 3.1.

Данные по теме «набор данных *medicines.txt*» приведены в таблице 2.1.

Таблица 2.1 – Набор данных *medicines.txt*

Лекарство	Симптомы
Парацетамол	Головная боль, высокая температура, боль в горле, озноб
Ибупрофен	Боль в суставах, боль в мышцах, воспаление, высокая температура
Лоратадин	Насморк, чихание, зуд, слезотечение
Но-шпа	Спазм, боль в животе, боль в желудке
Активированный уголь	Тошнота, вздутие, отравление, диарея
Амброксол	Кашель, мокрота, бронхит

Программа сопоставляет введенные ключевые слова с симптомами каждой строки таблицы и выбирает лекарство с наибольшим числом совпадений.

2.5 Программа

За основу взята заготовка из методических указаний: окно *Frame* с текстовой областью, кнопками *Search* и *Exit*. Обработчик кнопки *Search* разбивает введенный текст по запятым на ключевые слова, читает список лекарств из файла, для каждой строки подсчитывает число совпадений ключевых слов с симптомами, выводит название и число совпадений в текстовую область и открывает в браузере *html*-документ лекарства с наибольшим числом совпадений. Основные методы приведены в приложении А.

Чтение файла через *URL* и потоковые классы, подсчет совпадений и открытие браузера выполняют методы, приведенные в следующем листинге.

2.6 Результаты работы

Окно приложения после запуска показано на рисунке 2.1.

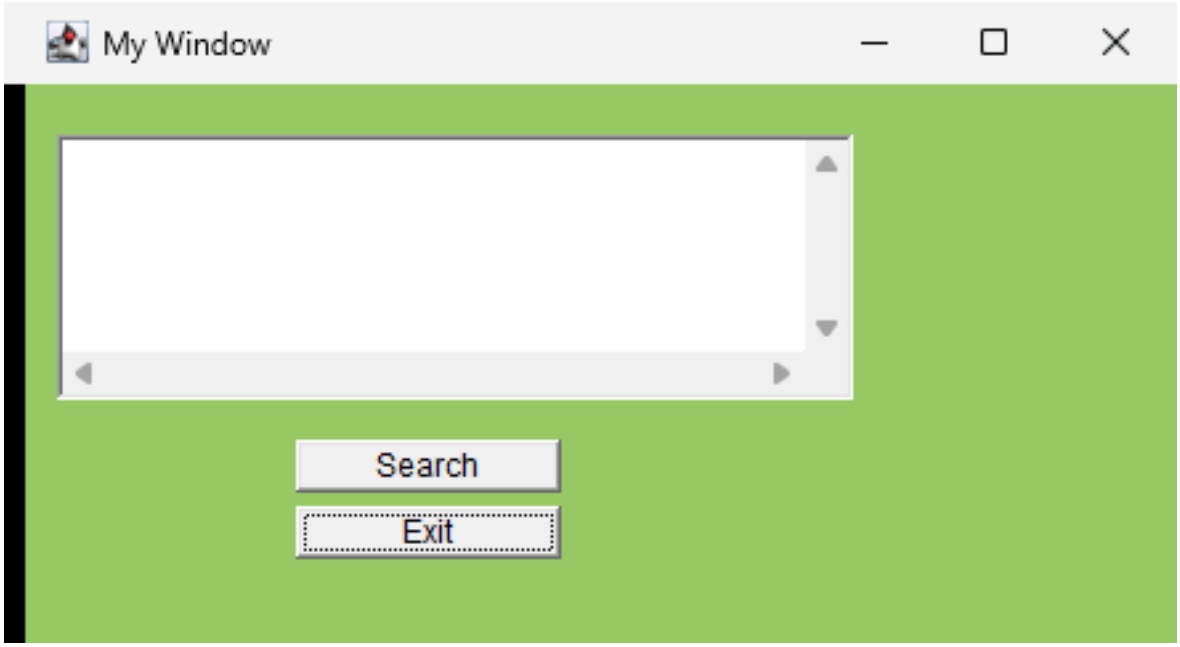


Рисунок 2.1 – Окно приложения после запуска

Окно содержит пустую текстовую область, кнопку поиска *Search* и кнопку выхода *Exit*.

В текстовую область введены симптомы «головная боль, высокая температура», как показано на рисунке 2.2.

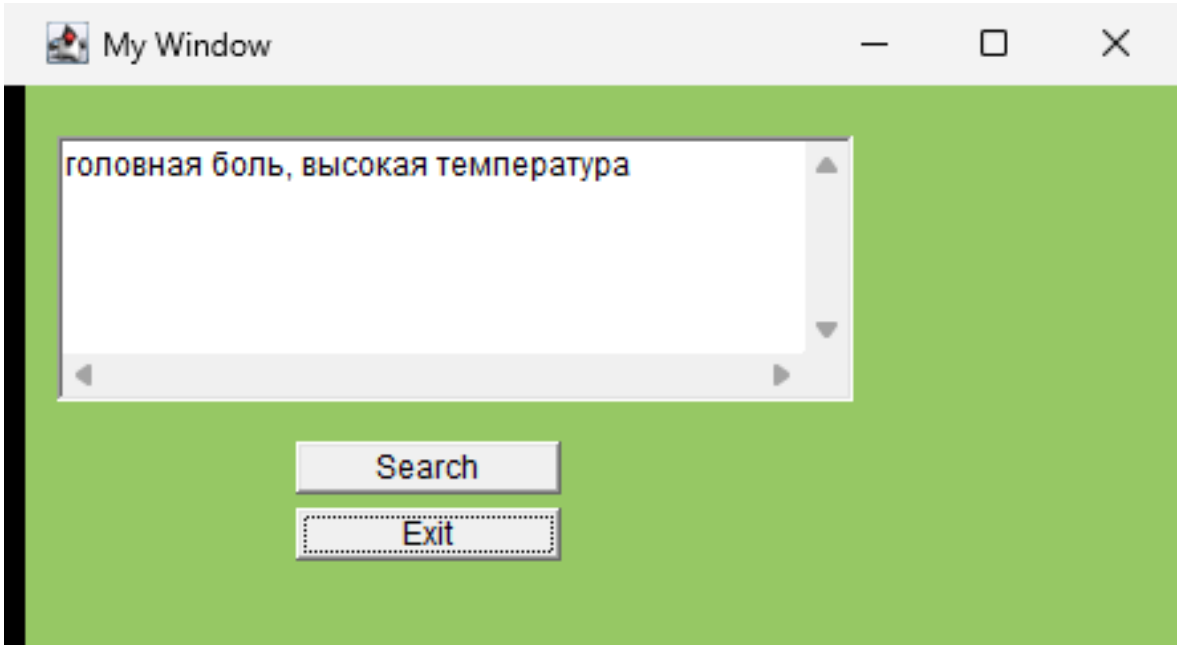


Рисунок 2.2 – Ввод ключевых слов

Ключевые слова разделены запятыми, их число находится в пределах от одного до трех, как требует вариант.

Результат поиска показан на рисунках 2.3 и 2.4.

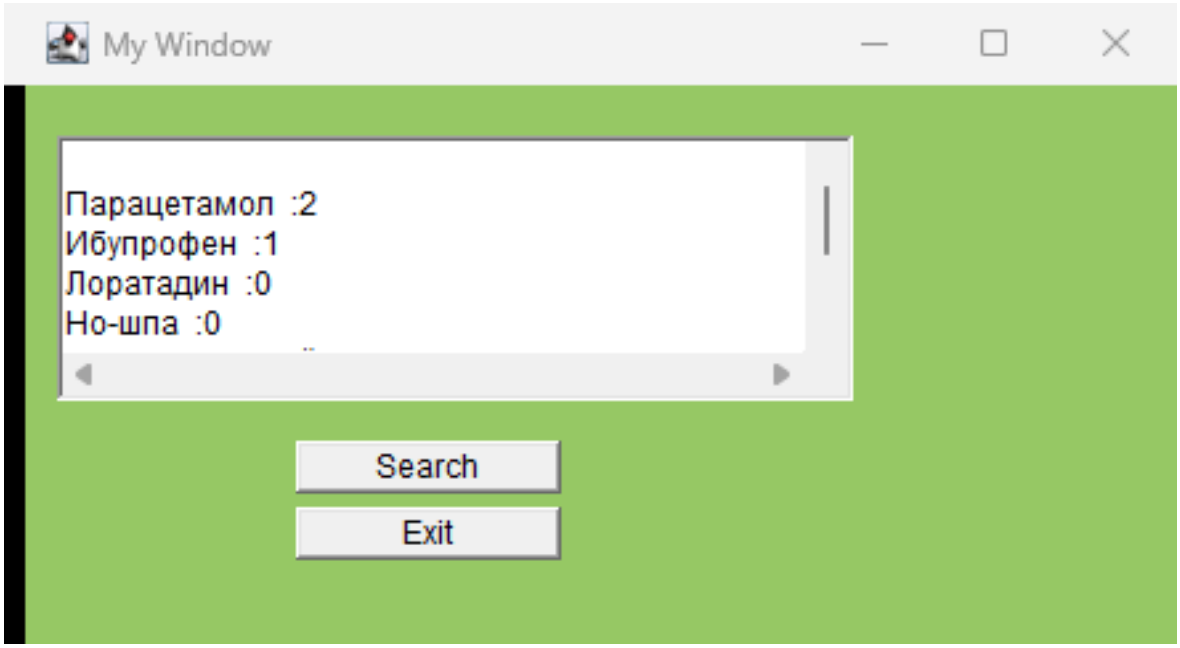


Рисунок 2.3 – Результат поиска (начало списка)

Для лекарства «Парацетамол» найдено 2 совпадения, для «Ибупрофена» – 1, для «Лоратадина» и «Но-шпы» – 0.

Конец списка с найденным лекарством показан на рисунке 2.4.

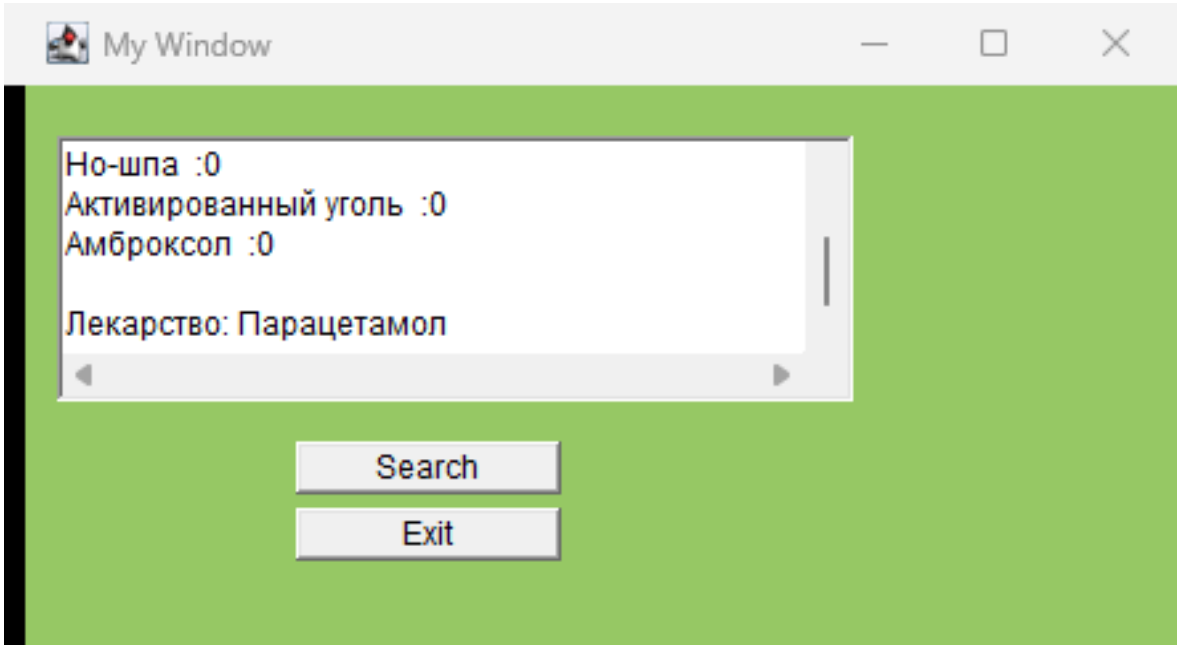


Рисунок 2.4 – Результат поиска (конец списка)

Список завершается строкой «Лекарство: Парацетамол», то есть выбран документ с наибольшим числом совпадений.

Найденный *html*-документ открывается в браузере, как показано на рисунке 2.5. Снимок страницы получен отдельно, чтобы показать содержимое документа.

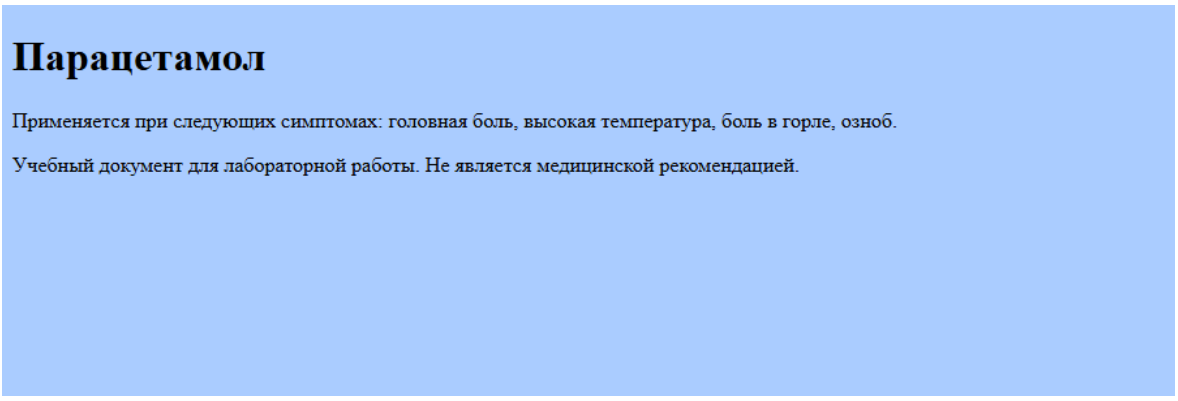


Рисунок 2.5 – Документ лекарства в браузере

В браузере открыта страница «Парацетамол» с перечнем симптомов, при которых применяется лекарство.

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое *URL*?

URL (*Uniform Resource Locator*) – унифицированный адрес ресурса, состоящий из протокола, адреса узла и пути, например *file*:///*C*:/*work/page.html* или *http://localhost*:8080/*index.html*. В программе файл превращается в *URL* методом *file.toURI*().*toURL*().

2. Как программно получить список всех файлов в папке?

Метод *listFiles*() объекта *File*, созданного для каталога, возвращает массив файлов: *new File*("*source_html*").*listFiles*(). В заготовке методических указаний массив преобразуется в список *ArrayList<File>* и обрабатывается в цикле.

3. Как программно запустить браузер и открыть *html*-файл?

Метод *Desktop.getDesktop*().*browse*(*uri*) открывает указанный адрес в браузере по умолчанию. В программе вызывается *Desktop.getDesktop*().*browse*(*file.getAbsoluteFile*().*toURI*()).

4. Как используются потоковые классы в приложении?

Метод *URLConnection.getInputStream*() возвращает байтовый поток ресурса. Класс *InputStreamReader* преобразует байты в символы в кодировке *UTF-8*, класс *BufferedReader* читает текст построчно методом *readLine*(). Блок *try-with-resources* автоматически закрывает потоки после чтения.

ВЫВОДЫ

Изучены возможности поиска информации в *URL*-ресурсах: получение потока через *URLConnection*, построчное чтение файла, подсчет совпадений ключевых слов и открытие найденного документа в браузере. Заготовка из методических указаний доработана под вариант 2: поиск названия лекарства по набору симптомов из текстового файла. Приложение корректно определило лекарство с наибольшим числом совпадений. Поставленная цель достигнута.

ПРИЛОЖЕНИЕ А
(ОБЯЗАТЕЛЬНОЕ)
ЛИСТИНГИ ПРОГРАММНОГО КОДА

Листинг А.1 – Обработчик кнопки *Search*

```
public void actionPerformed(ActionEvent ae) {
    if (ae.getSource() == exitButton) {
        System.exit(0);
    } else if (ae.getSource() == searchButton) {
        // Ключевые слова (симптомы) разделяются запятыми
        String[] keywords = textArea.getText().split(",");
        textArea.setText("");

        // Список лекарств читается из текстового файла
        File medicinesFile = new File("medicines.txt");
        ArrayList<String> medicines = readLines(medicinesFile);

        String bestName = null;
        int bestCount = 0;
        for (String line : medicines) {
            // Формат строки: Название | симптом1, симптом2, ...
            String[] parts = line.split("\\|");
            if (parts.length < 2) {
                continue;
            }
            String name = parts[0].trim();
            int coincidenceCount = testSymptoms(parts[1], keywords);
            textArea.append("\n" + name + " :" + coincidenceCount);
            if (coincidenceCount > bestCount) {
                bestCount = coincidenceCount;
                bestName = name;
            }
        }

        if (bestName == null) {
            textArea.append("\nЛекарство не найдено");
        } else {
            textArea.append("\n\nЛекарство: " + bestName);
            openInBrowser(new File("source_html", bestName + ".html"));
        }
    }
}
```

Листинг А.2 – Чтение файла, подсчет совпадений и открытие браузера

```
// Чтение строк файла через URL и потоковые классы
private ArrayList<String> readLines(File file) {
    ArrayList<String> lines = new ArrayList<>();
    try {
        URL url = file.toURI().toURL();
        URLConnection connection = url.openConnection();
```

```
        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(connection.getInputStream(), "UTF-8"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (!line.trim().isEmpty() && !line.startsWith("#")) {
                    lines.add(line);
                }
            }
        }
    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
    return lines;
}

// Число ключевых слов, найденных среди симптомов лекарства
private int testSymptoms(String symptoms, String[] keywords) {
    int results = 0;
    String content = symptoms.toLowerCase();
    for (String keyword : keywords) {
        if (!keyword.trim().isEmpty() && content.contains(keyword.trim().toLowerCase())) {
            results++;
        }
    }
    return results;
}

// Открытие html-документа в браузере
private void openInBrowser(File file) {
    try {
        Desktop.getDesktop().browse(file.getAbsoluteFile().toURI());
    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```